Leibniz FH

Wirtschaftsinformatik (B. Sc.)
Modul 20.1 (Vertiefung Entwicklung)
Sommersemester 2026

Anwendung erweiterter Softwareentwicklung

6. Parallelisierung und Persistenz

Dr. Peter Lohmann

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Leibniz FH

6.1. Nebenläufigkeit

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Motivation

- ✓ Viele reale Aufgaben sind von Natur aus nicht streng sequenziell
  - mehrere Nutzer greifen gleichzeitig auf einen Server zu
  - ein Programm wartet auf Netzwerkantwort und soll trotzdem reaktionsfähig bleiben
  - rechenintensive Aufgaben sollen mehrere Prozessorkerne nutzen
- ✓ Nebenläufigkeit erlaubt die Modellierung ineinander verzahnter Abläufe
- ✓ Darüber hinaus: Moderne Hardware bietet echte Parallelität über mehrere Kerne

Parallelität vs. Nebenläufigkeit

- ✓ Nebenläufigkeit (Concurrency)
  - mehrere Abläufe sind logisch gleichzeitig aktiv
  - muss nicht bedeuten, dass sie physisch zur selben Zeit laufen
- ✓ Parallelität (Parallelism)
  - mehrere Abläufe laufen tatsächlich gleichzeitig auf verschiedenen CPU-Kernen

Grundbegriffe

- ✓ Prozess
  - eigenständige Ausführungseinheit mit eigenem Speicher-Adressraum und Zustand
  - vom Betriebssystem verwaltet und isoliert
- ✓ Thread
  - leichtgewichtiger Ausführungsstrang innerhalb eines Prozesses
  - mehrere Threads teilen sich den gemeinsamen Adressraum des Prozesses
  - geringerer Aufwand bei Erstellung und Kontextwechsel als bei Prozessen
  - aber immer noch (für gewöhnlich) vom Betriebssystem verwaltet
- ✓ Coroutine
  - *kooperativ* schaltende Ausführungseinheit innerhalb eines einzelnen Threads
  - vom Coroutine-Framework verwaltet, nicht vom Betriebssystem

Leibniz FH

6.2. Modelle in Python

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Threads

- ✓ Direkter gemeinsamer Zugriff auf Objekte im Heap
- ✓ Gut geeignet für I/O-lastige Aufgaben
  - Warten auf Netzwerkantworten
  - Lesen und Schreiben von Dateien
- ✓ Einschränkung durch den GIL
  - *Global Interpreter Lock* blockiert echte parallele Ausführung in CPython
  - Aber C-Extensions können den GIL „umgehen“, z.B. numpy

Prozesse

- ✓ Keine gemeinsame Nutzung von Speicherobjekten
- ✓ Ermöglicht echte Parallelität auf mehreren Kernen
  - kein GIL-Problem, da jeder Prozess seinen eigenen Interpreter hat
- ✓ Kommunikation zwischen Prozessen erfordert explizite Mechanismen
  - Queues, Pipes oder Shared Memory
  - höherer organisatorischer Aufwand als bei Threads

Coroutines

- ✓ Kooperative Nebenläufigkeit mit **`async/await`**
  - eine Coroutine gibt die Kontrolle *ausschließlich explizit* an andere ab
- ✓ Gut geeignet für sehr viele gleichzeitig wartende Aktivitäten
  - tausende offene Netzwerkverbindungen ohne Blockieren
- ✓ Alles läuft innerhalb eines einzigen Betriebssystem-Threads
  - kein echter Parallelismus, aber effiziente (selbst-regulierende) Verschachtelung
  - ähnliche Vorteile wie Threads, aber noch effizienter
- ✓ Erfordert konsequente Umsetzung auf allen Abstraktionsebenen
  - aufwändiger in der initialen Programmierung
  - einfacher in der späteren Verwaltung

Leibniz FH

6.3. Parallele Ausführung

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

threading.Thread

- ✓ Direktes Erstellen und Starten von Threads
- ✓ Typisches Muster
  - Thread-Objekt mit Zielfunktion erzeugen
  - mit `start()` starten
  - mit `join()` auf Beendigung warten

concurrent.futures

- ✓ Höheres Abstraktionsniveau über das Executor-Konzept
- ✓ Zwei zentrale Klassen
  - **`ThreadPoolExecutor`** für Thread-basierte Parallelität
  - **`ProcessPoolExecutor`** für echte CPU-Parallelität mit separaten Prozessen
- ✓ Aufgaben werden als **`Future`**-Objekte verwaltet
  - Ergebnis kann später abgerufen werden
  - Ausnahmen werden sauber weitergeleitet

asyncio

- ✓ Asynchrone Ausführung mit Coroutines und Tasks
- ✓ Grundlegende Konzepte
  - `async def` definiert eine Coroutine-Funktion
  - `await` übergibt die Kontrolle und wartet auf ein Ergebnis
  - `asyncio.Task` kapselt eine laufende Coroutine
- ✓ Geeignet für viele gleichzeitige Warteoperationen ohne Threads

Leibniz FH

6.4. Synchronisation

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Konkurrierende Zugriffe

- ✓ Mehrere Threads, die gleichzeitig auf gemeinsame Daten zugreifen, können sich gegenseitig stören
- ✓ *Race Condition*
  - das Ergebnis hängt von der zufälligen Reihenfolge der Zugriffe ab
  - nicht reproduzierbar und schwer zu testen
- ✓ Inkonsistente Zustände
  - ein Thread liest Daten, während ein anderer sie gerade verändert
  - das Ergebnis ist unvorhersehbar und oft falsch
- ✓ Atomare Operationen
  - können nicht von anderen Threads unterbrochen werden
  - reichen manchmal bereits aus, um Race Conditions zu vermeiden
    - z.B. `dct[key] = value`

Lock

- ✓ `threading.Lock` stellt gegenseitigen Ausschluss sicher
  - genau ein Thread kann den Lock gleichzeitig halten
  - andere Threads warten, bis er freigegeben wird
- ✓ `threading.RLock` erlaubt rekursives Sperren durch denselben Thread
  - nützlich, wenn gesperrter Code gesperrte Methoden aufruft

Event

- ✓ `threading.Event`
  - einfaches Signal zwischen Threads
  - ein Thread setzt das Event, andere warten darauf

Semaphore

- ✓ `threading.Semaphore`
  - begrenzt die Anzahl gleichzeitig aktiver Zugriffe
  - nützlich zur Ressourcenbegrenzung

Queue

- ✓ `queue.Queue`
  - Thread-sichere Warteschlange für die Übergabe von Aufgaben und Daten
  - Intern synchronisiert
    - kein eigenes Locking notwendig
  - Typisches Muster: Producer-Consumer
- ✓ Konzept ist auch für multiple Processes nützlich

Leibniz FH

6.5. Typische Probleme

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Deadlocks

- ✓ Zwei oder mehr Threads warten gegenseitig aufeinander und kommen nicht mehr weiter
- ✓ Klassisches Szenario
  - Thread A hält Lock 1 und wartet auf Lock 2
  - Thread B hält Lock 2 und wartet auf Lock 1
- ✓ Keine Seite kann ohne die andere weitermachen
- ✓ Vermeidung
  - Locks immer in derselben Reihenfolge anfordern
  - RLock oder höhere Abstraktionen bevorzugen

Starvation

- ✓ Ein Thread bekommt dauerhaft keine Rechenzeit, obwohl er bereit ist
- ✓ Ursache
  - andere Threads werden systematisch bevorzugt
  - Lock wird immer von denselben Threads gehalten
- ✓ Folge
  - bestimmte Aufgaben werden nie oder kaum erledigt
  - schwer zu reproduzieren und zu diagnostizieren

Nichtdeterminismus

- ✓ Nebenläufige Programme sind nicht deterministisch
  - die Reihenfolge, in der Threads laufen, ist nicht vorhersehbar
  - dieselbe Eingabe kann zu unterschiedlichen Abläufen führen
- ✓ Fehler können sehr selten und nur unter bestimmten Timing-Bedingungen auftreten
  - *Heisenbugs*: verschwinden beim Beobachten oder Debuggen

Erschwertes Debugging

- ✓ Klassische Debugger sind bei Nebenläufigkeit eingeschränkt
  - Haltepunkte verändern das Timing und beeinflussen das Verhalten
- ✓ Hilfreiche Techniken
  - strukturierte Logs mit Thread-ID und Zeitstempel
  - Reproduzierbare Testszenarien mit kontrollierten Abläufen
- ✓ Grundregel
  - je weniger geteilter Zustand, desto einfacher das Debugging

Leibniz FH

6.6. Serialisierung

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Was ist Serialisierung?

- ✓ Serialisierung bezeichnet die Umwandlung von Objekten in eine speicher- oder übertragbare Form
- ✓ Deserialisierung ist der umgekehrte Vorgang
- ✓ Einsatzgebiete
  - Objekte persistent in Dateien speichern
  - Daten über Netzwerk übertragen - auch localhost
- ✓ Beispiele
  - Textformate: XML, JSON
  - Universelle Binäformate: MessagePack, Protobuf
  - Spezielle Binärformate: DB-spezifische Formate, Sprach-spezifische Formate

Pickle

- ✓ Python-spezifisches Binärformat
- ✓ Kann fast beliebige Python-Objekte serialisieren
  - eigene Klassen, Funktionen, komplexe Strukturen
- ✓ Sicherheit
  - `pickle.load()` auf nicht vertrauenswürdige Daten kann beliebigen Code ausführen!
  - nur in kontrollierten Umgebungen einsetzen!